Faculty of Engineering & Technology

Department of Computer Science

DATA BASE SYSTEMS - Comp 333

Prepared by:

Samir Ali

Business Model

Mavi Palestine is the official local distributor of Mavi Jeans, a well-known international fashion brand specializing in denim wear and casual clothing. The company focuses on offering high-quality jeans, shirts, jackets, and accessories for men and women. These products are imported from an international supplier in Turkey and stored in the main warehouse before being sent to the main store. So, in other words, it currently operates one retail store and a central warehouse.

Client's operations

They are centred around three main processes:

1. **Purchasing:**
   The company imports products from Mavi's global supplier. Each shipment includes various product categories, sizes, and styles.
2. **Sales:**
   The retail store sells products directly to customers. Sales transactions are recorded daily and influence stock levels in the store and the warehouse.
3. **Warehouse & Inventory Management:**
   The company stores large quantities of stock in the main warehouse and refills products in the store as needed. Maintaining accurate records of quantities, stock movement.

Client difficulties

Currently, Mavi Palestine relies on a mix of manual tracking and simple spreadsheets to monitor product availability, sales performance, and supplier orders. This approach often leads to:

- Difficulty in tracking real-time stock levels
- Errors in recording sales or purchases
- Inefficient reporting on best-selling products or slow-moving inventory

The client requires a centralized database management system to:

- Store and organize all product, supplier, and sales information

- Automate reporting and data retrieval
- Manage purchase orders and supplier deliveries
- Monitor inventory levels in the store and warehouse

Project Scope and Description

Project Title:

Mavi Palestine Sales and Inventory Management System

Project Scope

This project aims to design and develop a database-driven sales and inventory management system for Mavi Palestine, the proposed System will maintain structured data for all products, supplier, store, employees, and transactions. The system will store comprehensive details about each entity, including product categories, pricing, quantities, supplier information, and customer purchases, and will link them through well-defined relationships. The system will serve as a centralized database that connects the store with the warehouse, ensuring real-time access to data such as product availability, supplier information, and sales performance. The project will focus on:

- Product Management: Maintain detailed records for each product (category, size, color, price, stock quantity, etc.).
- Inventory and Warehouse Management: Track the movement of products between warehouse and store; update stock levels automatically after each sale or delivery.
- Sales Management: Record customer purchases, calculate totals, and store transaction history for the store.
- Supplier Management: Store supplier information, manage purchase orders, and track delivery dates.

Entities and Relationships

The Mavi Palestine Sales and Inventory Management System will rely on several core entities that represent the company's daily operations — including product management, sales, purchasing, supplier, employees, and warehouse tracking. Each entity will later become a relational table in the database, with well-defined relationships linking them.

1. Entities

Entity Name	Description	Key Attributes
Store	Represents the physical retail store where sales take place.	Store_ID (PK), Store_Name, Location
Employee	Records details of employees working in the store.	Employee_ID (PK), First_Name, Last_Name, Store_ID (FK), Role, Username, Password
Supplier	Stores information about Mavi's external supplier.	Supplier_ID (PK), Supplier_Name, Contact_Person, Phone, Email
Product	All products sold by Mavi Palestine.	Product_ID (PK), Product_Name, Size, Default_Sale_Price, Reorder_Level, Supplier_ID(FK), Category, Unit_Cost
Warehouse	Represents the central that warehouse stores inventory.	Warehouse_ID (PK), Location
Inventory	Tracks product quantities at each physical location (warehouse or store) using separate records.	Inventory_ID (PK), Product_ID (FK), Warehouse_ID (FK) Store_ID (FK) Quantity_Available

Purchase	Represents purchase orders sent to the supplier.	Purchase_ID (PK), Supplier_ID (FK), Warehouse_ID (FK), Purchase_Date Employee_ID (FK),
Purchase_Details	Itemized list of products in each purchase order.	PurchaseDetails_ID (PK), Purchase_ID (FK), Product_ID (FK), Quantity, Unit_Cost,
Customer	Stores customer information for sales tracking.	Customer_ID (PK), First_Name, Last_Name, Phone
Sale	Records customer purchase transactions at the store.	Sale_ID (PK), Customer_ID (FK), Employee_ID (FK), Store_ID (FK), Sale_Date Sale_Payment_method
Sale_Details	Line items for each sale (multiple products per receipt).	SaleDetails_ID (PK), Sale_ID (FK), Product_ID (FK), Quantity, Unit_Price
Stock_transfer	Represents one transfer transaction	Transfer_ID (PK), Warehouse_ID (FK), Store_ID(FK), Transfer_Date, Employee_ID(FK)
Stock_transfer_details	Represents the individual products and quantities inside a transfer.	TransferDetails_ID(PK), Transfer_ID(FK), Product_ID(FK), Quantity_Transferred
Payment	Represents the payment done by the store (from the employee) to the supplier and there could be multiple payments on a specific order (an order is paid partially eached payment)	Payment_ID(PK), Purchase_ID(FK), Payment_Method, Payment_Amount, Payment_Date

2. Relationships Between Entities

Relationship	Description	Type
Store – Employee	A store employs many employees; each employee belongs to one store.	One-to-Many
Store – Sale	All sales occur in a specific store; each store can have many sales.	One-to-Many
Store – Inventory	The store has inventory records for each product. (so many inventory records)	One-to-Many
Warehouse – Inventory	The warehouse has inventory records for each product.	One-to-Many
Supplier – Purchase	One supplier can have many purchase orders; each purchase is from only one supplier.	One-to-Many
Purchase – Purchase_Details	Each purchase order contains multiple pusrchase details.	One-to-Many
Product – Purchase_Details	A product can appear in multiple purchase detail rows.	One-to-Many
Product – Inventory	A product can have multiple inventory entries (store and warehouse).	One-to-Many
Customer – Sale	A customer can make multiple sales; each sale belongs to one customer.	One-to-Many
Employee – Sale	An employee handles many sales; each sale is handled by one employee.	One-to-Many
Sale – Sale_Details	Each sale can contain multiple line items.	One-to-Many
Product – Sale_Details	A product can appear in many sale detail rows.	One-to-Many

Warehouse – Stock_Transfer	A warehouse can send many transfers; each transfer is from one warehouse	One-to-Many
Store – Stock_Transfer	A store can receive many transfers; each transfer is to one store	One-to-Many
Stock_Transfer – Stock_Transfer_Details	A stock transfer consists of many transfer detail lines	One-to-Many
Product – Stock_Transfer_Details	A product can be moved in many transfers; each transfer line is for one product.	One-to-Many
Employee – Stock_Transfer	An employee can record many transfers; each transfer is recorded by one employee	One-to-Many
Payment – purchase	Each purchase order should be paid for and each purchase could have multiple payements	One – to – many

At a high level, the database will connect:

Supplier → Purchases → Warehouse → store → Sales → Customers

This logical flow reflects the complete business process at Mavi Palestine from product receival to final sale and ensures that all data remains consistent and traceable throughout the company's operations.

Sample Queries Demonstrating Database Functionality

1. Retrieve all purchase orders made during the current year.
2. Retrieve the total quantity of products received into the warehouse for a month.
3. Retrieve the total purchase cost per month for a given year.
4. Retrieve the total quantity purchased for each product during the current year.

5. Retrieve the products with the highest total purchase cost during the current year.
6. Retrieve the list of products that are currently below the reorder level in the warehouse.
7. Retrieve all purchase orders that have no payments yet.
8. Retrieve products that have never been purchased.
9. Retrieve all stock transfers made within a specific date range (with warehouse, store, and total items transferred).
10. Retrieve the current inventory valuation per warehouse.
11. Retrieve all employees' full names and positions and username
12. Retrieve all products with their category, size, unit price, and available quantity in both store and warehouse.
13. Retrieve the top five best-selling products during a selected month and year.
14. Retrieve all sales handled by a specific employee, showing sale date, customer name, and total amount
15. Retrieve unsold products along with their name, size, category, unit price.
16. Retrieve monthly sales summary based on a given month and year showing the total sales, total revenue, average sale.
17. Retrieve bulk sales (sales that contains 10 or more items).
18. Retrieve sales transactions done by each employee (choose the employee).
19. Retrieve the inventory overview showing the store and warehouse inventory with the status (available, low, out).
20. Retrieve the customers who have more made more than three purchases in total.

Technologies intended to be used

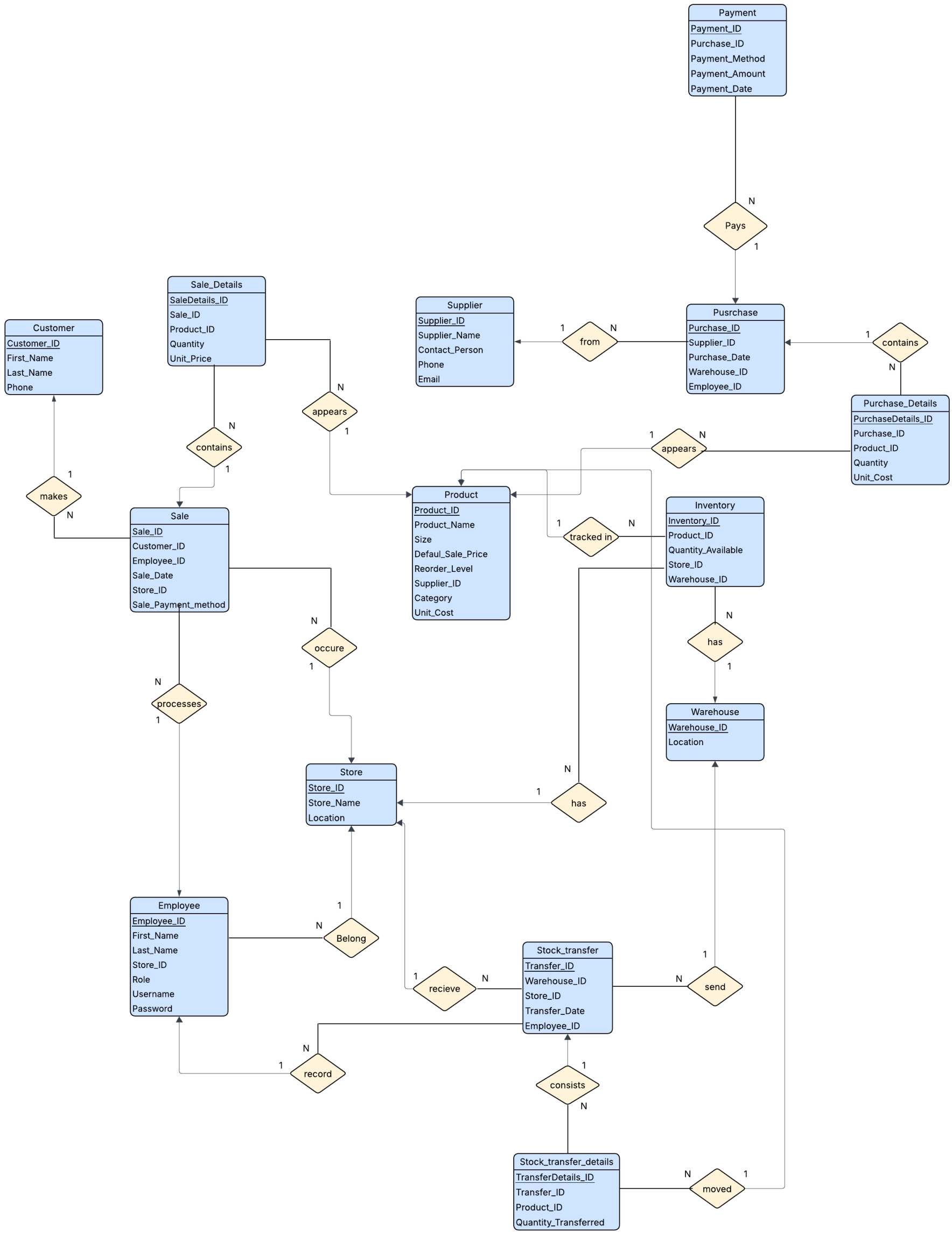
For the implementation of this project, we will use **Python** as the primary programming language due to its simplicity, flexibility, and strong database integration support. The system will be developed using the **Flask** web framework, which provides a lightweight and efficient structure for building dynamic web applications. Flask will handle backend operations such as database connectivity, query execution, and data management using MySQL. The front end will be designed using **HTML and CSS** to create a clean and user-friendly interface that allows users to interact easily with the system through their web browser. A small amount of **JavaScript** is used within

the HTML code to enhance the user interface, improve interactivity, and provide basic client-side validation and visual feedback.

3NF Normalization

- The Purchase_Subtotal and Sale_Subtotal attributes were removed from the Purchase and Sale table because it is a derived value that can be calculated from the Purchase_Details and Sale_Details tables. Storing this attribute would introduce redundancy and violate Third Normal Form. Instead, the subtotal is computed dynamically using a query, ensuring consistency and full compliance with 3NF.
- The database is in Second Normal Form because it is already in First Normal Form and none of its tables use a composite primary key. All relations are defined with single-attribute primary keys, which means that partial dependency of non-key attributes on a subset of a primary key cannot occur. Consequently, every non-key attribute in each table is fully functionally dependent on its entire primary key. This eliminates the possibility of partial dependencies and ensures that the database satisfies the requirements of 2NF.
- The database is in Third Normal Form because it is already in Second Normal Form and contains no transitive dependencies among non-key attributes. In every table, all non-key attributes depend directly and exclusively on the primary key. Descriptive and entity-specific information—such as customer details, employee information, supplier data, product attributes, and store information—is stored only in its corresponding table and is referenced elsewhere using foreign keys rather than being duplicated. No non-key attribute depends on another non-key attribute, which prevents redundancy and update anomalies. As a result, the database satisfies the requirements of Third Normal Form.

ER Diagram


Payment
Payment_ID
Purchase_ID
Payment_Method
Payment_Amount
Payment_Date
Pays
N
1
Pusrchase
Purchase_ID
Supplier_ID
Purchase_Date
Warehouse_ID
Employee_ID
contains
Purchase_Details
PurchaseDetails_ID
Purchase_ID
Product_ID
Quantity
Unit_Cost
Supplier
Supplier_ID
Supplier_Name
Contact_Person
Phone
Email
from
Customer
Customer_ID
First_Name
Last_Name
Phone
Sale_Details
SaleDetails_ID
Sale_ID
Product_ID
Quantity
Unit_Price
appears
contains
makes
Sale
Sale_ID
Customer_ID
Employee_ID
Sale_Date
Store_ID
Sale_Payment_method
Product
Product_ID
Product_Name
Size
Defaul_Sale_Price
Reorder_Level
Supplier_ID
Category
Unit_Cost
tracked in
Inventory
Inventory_ID
Product_ID
Quantity_Available
Store_ID
Warehouse_ID
has
Warehouse
Warehouse_ID
Location
occure
processes
Store
Store_ID
Store_Name
Location
has
Employee
Employee_ID
First_Name
Last_Name
Store_ID
Role
Username
Password
Belong
recieve
Stock_transfer
Transfer_ID
Warehouse_ID
Store_ID
Transfer_Date
Employee_ID
send
record
consists
Stock_transfer_details
TransferDetails_ID
Transfer_ID
Product_ID
Quantity_Transferred
moved